# Modificando Recetas

Descargar estos apuntes

## Ejercicio

A partir del ejercicio de recetas ya implementado en este bloque, vamos a modificarlo de forma que ahora podremos visualizar más de una receta utilizando un componente especifico para ello.
Primero deberemos modificar levemente el mock Recipe para que photo pase a ser un String? (que corresponderá al nombre de la imagen), deberemos de hacer el mismo cambio en el resto de elementos que representan a Recipe en las distintas partes de la arquitectura.
Lo siguiente será crear en RecipeDaoMock una lista de recetas en vez de una sola. Los datos para estas podrán ser los que se muestran a la derecha.

Como es de esperar, tendrás que cambiar RecipeViewModel de forma que ahora recipeState pasará a ser una lista de recetas extraídas del repositorio. Tendremos que modificar el método del evento, para que nos entre la receta sobre la que se ha hecho click, y en la funcionalidad deberemos trabajar sobre la receta pulsada de la lista.

```
private var recipes = mutableListOf(
    RecipeMock(
        id = 1,
        name = "Magdalenas de la abuela",
        description = "Fabulosas magdalenas con pepitas de chocolate y un suave sabor a naranja.",
        chef = "Carlos Arguiñano",
        urlPhoto = "http://10.0.2.2/recetas/magdalenas.png",
        numLikes = 8,
        iLike = false
    ),
    RecipeMock(
        id = 2,
        name = "Pan casero con tomates secos",
        description = "Novedoso pan casero con un estimulante sabor a tomates secos, perfecto para acompañar platos
        chef = "Carlos Arguiñano",
        urlPhoto = "http://10.0.2.2/recetas/pan.png",
        numLikes = 3,
        iLike = false
    ),
    RecipeMock(
        id = 3,
        name = "Ensalada multicolor con nueces",
        description = "Receta sencilla y rápida que dejará impresionados a todos tus invitados.",
        chef = "Sonia de la Oz",
        urlPhoto = "http://10.0.2.2/recetas/ensalada.png",
        numLikes = 5,
        iLike = false
    ),
    RecipeMock(
        id = 4,
        name = "Caldo de almendras, pasas y calabaza",
        description = "Estupenda sopa vejetal de invierno, con un sabor exótico inmejorable",
        chef = "Mohamed Aldalay",
        urlPhoto = null,
        numLikes = 12,
        iLike = false
    )
)
```

En cuanto a la parte de la vista, que es en la que vamos a realizar más cambios, vamos a crear una función composable nueva **CardRecipe** que tendrá el código que hasta ahora teníamos en RecipeScreen (ya que a esta función le llegará una sola receta y el evento sobre esta, que ya habíamos establecido en el anterior ejercicio). Como ahora las recetas tendrán distintas imágenes, tendrás que extraerlas de los **recursos** como hemos hecho en otros ejercicios y controlando si la imagen es null para asignar la imagen por defecto que podremos encontrar en recursos `local_dining_24px`.

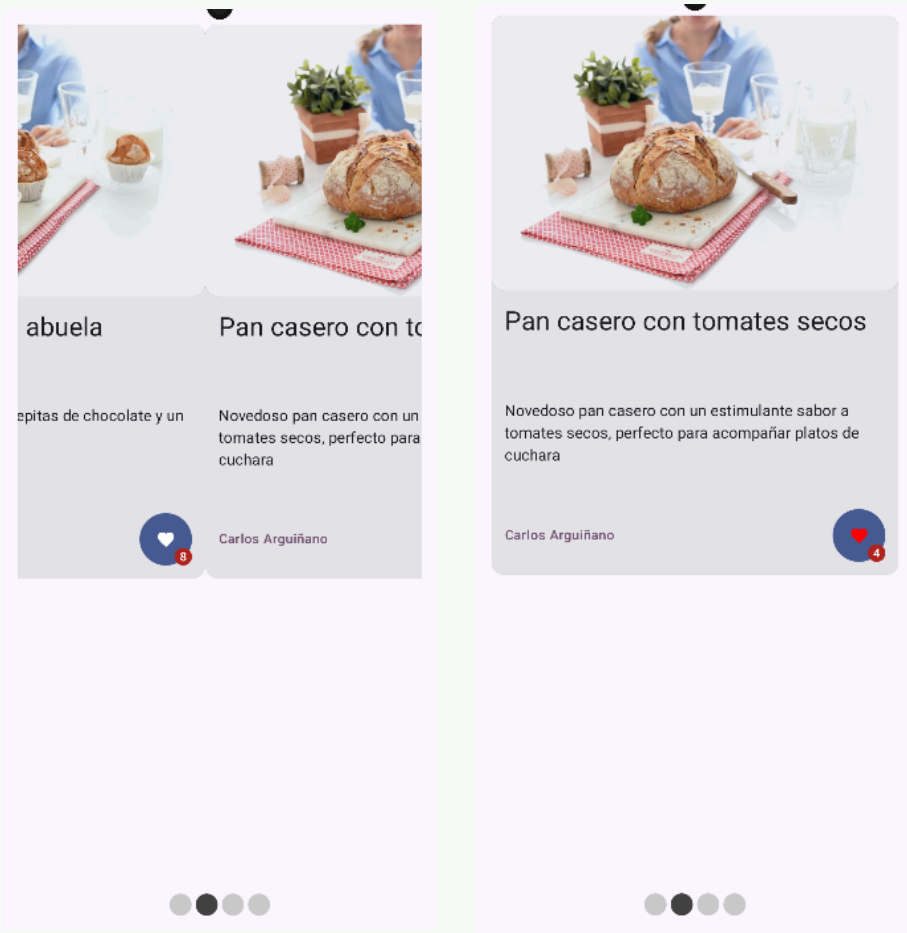
En cambio a la función `RecipeScreen` le llegará la lista de las recetas y ese mismo evento, y llamará a otra `CardRecipe` con la receta correspondiente.

Para realizar esto tendremos que añadir un componente **localizador** **HorizontalPager** que nos permitirá la paginación entre las recetas con la llamada dentro de este a `CardRecipe` con la receta que corresponda a

la página en transcurso (page 0 sería la primera). Conviene añadir este composable dentro de un Box que ocupe el máximo de alto.

Además añadiremos un **Indicador de página** que nos mostrará la página que estamos mostrando en cada momento como indicativo de las recetas que se pueden visualizar.

> **Nota**
>
> Estudia atentamente la documentación de **HorizontalPager** y **El indicador de página**, en los enlaces proporcionados, para ver como se implementa y que parámetros son necesarios, además de los imports que deberás añadir a tu proyecto.




Vamos a implementar dos **ViewModels** haciendo que el evento de **ILike** reciba la receta sobre la que se ha hecho click y así saber qué receta modificar.

```
sealed interface RecipeEvent {
    data class OnClickButtonLike(val recipe: RecipeUiState) : RecipeEvent
}
```

Abordando el problema de dos maneras distintas:

## Opción 1: RecipeViewModel1.kt con List de recetas

Con una lista inmutable que tras actualizar la receta recibida en el repositorio, se vuelve a cargar toda la lista de recetas para actualizar la vista.

```
private val repository: RecipeRepository = RecipeRepository()

var recipesState by mutableStateOf<List<RecipeUiState>>(listOf())
init {
    recipesState = repository.get().map { it.toRecipeUiState() }
}
```

Este enfoque nos permite trabajar con una lista inmutable, pero cada vez que se modifica una receta, rebujando todos los cards en la misma.

## Opción 2: RecipeViewModel2.kt con SnapshotStateList de recetas

Este tipo de lista mutable con estado ( `SnapshotStateList` ), nos permite modificar el **objeto receta completo** que se ha pulsado sin tener que volver a cargar toda la lista de recetas, por lo que es más eficiente.

> ⚠ **Aviso**
>
> Ojo hay que modificar la receta completa, no solo la propiedad con número de likes, ya que si modificamos solo el número de likes, la receta no se volverá a dibujar en la vista.

Pero para ello, tendremos que buscar el objeto receta que se ha pulsado en la lista y sustituirlo por el nuevo objeto receta que nos devuelve el repositorio tras la actualización.

Fíjate que definimos el `SnapshotStateList` a partir de la función `mutableStateListOf` , que nos permite crear una lista mutable con estado, y que podemos inicializar con una lista de recetas obtenida del repositorio que transformamos de inmutable a mutable con estado mediante la función `toMutableStateList()` .

```
private val repository: RecipeRepository = RecipeRepository()
var recipesState = mutableStateListOf<RecipeUiState>()

init {
    recipesState = repository.get().map { it.toRecipeUiState() }.toMutableStateList()
}
```